valutare interfacce

metodologie di programmazione

samory@di.uniroma1.it

📣 annunci

- Form prenotazione appelli
- Consegna progetto tramite classroom
- Prepararsi
- OPIS

OPIS

Link alle istruzioni:
https://www.uniroma1.it/sites/default/files/field_file_allegati/vademecum_per_studenti_opis_2023_24_1.pdf

Codice del corso: 78MADF8Q

🎯obiettivi

ho fatto il diagramma delle classi. è corretto?

Qualità dell'Interfaccia di una Classe

- Clienti: Programmatori che utilizzano la classe
- Criteri:
  - Coesione
  - Completezza
  - Comodità
  - Chiarezza
  - Coerenza
- Attività di ingegneria: fare compromessi

Coesione

- La classe descrive un'**unica astrazione**
- I metodi devono essere correlati all'unica astrazione
- Esempio negativo:

```
public class Mailbox

{

   public addMessage(Message aMessage) {
... }

   public Message getCurrentMessage() {
... }

   public Message removeCurrentMessage()
{ ... }

   public void processCommand(String
command) { ... }

   ...

}
```

Completezza

- **Supporta operazioni** che sono attese e ben definite **per l'astrazione**
- Esempio potenzialmente negativo: Date

```
Date start = new Date();

// fai cose

Date end = new Date();
```

- Quanti millisecondi sono trascorsi?

- Nessuna operazione del genere nella classe Date
- Rientra nelle sue responsabilità?
- Dopotutto, abbiamo before, after, getTime

Comodità

- Una buona interfaccia rende **tutte** le attività **possibili** ... e le attività **comuni semplici**
- Esempio negativo: Lettura da System.in prima di Java 5.0

```
BufferedReader in = new
BufferedReader(new
InputStreamReader(System.in));
```

- Perché System.in non ha un metodo readLine?
- Dopotutto, System.out ha println.
- La classe Scanner risolve l'inconveniente

Chiarezza

- Programmatori confusi scrivono codice buggato
- Esempio negativo: Rimozione di elementi da LinkedList
- Reminder: Classe standard della lista collegata

```
LinkedList<String> countries =
new LinkedList<String>();

countries.add("A");
countries.add("B");
countries.add("C");
```

Iterazione sulla lista:

```
ListIterator<String> iterator =
countries.listIterator();

while (iterator.hasNext())

   System.out.println(

    iterator.next());
```

Chiarezza

- L'iteratore è posizionato **tra** gli elementi
- Come il cursore in un elaboratore di testi
- add aggiunge a sinistra dell'iteratore (come in un elaboratore di testi):
- Aggiungi X prima di B:

```
ListIterator<String> iterator =
countries.listIterator(); // |ABC

iterator.next(); // A|BC

iterator.add("France"); // AX|BC
```

Chiarezza

- Per rimuovere i primi due elementi, non puoi semplicemente "cancellare"
- remove **non rimuove l'elemento a sinistra** dell'iteratore
- *Removes from the list the last element that was returned by next or previous. This call can only be made once per call to next or previous. It can be made only if add has not been called after the last call to next or previous..*
- Eh?

```
ListIterator<String> iterator =
countries.listIterator(); // |ABC

iterator.next(); // A|BC

iterator.add("France"); // AX|BC
```

👐TPS

Quali principi della buona programmazione sono stati violati?

Coerenza

- Le **funzionalità correlate** di una classe devono avere nomi, parametri, valori di ritorno e **comportamenti corrispondenti**
- Esempio negativo:

```
new GregorianCalendar(year, month - 1, day)
```

- Perché il mese è basato su 0?

Coerenza

- Esempio negativo: classe String

```
s.equals(t) / s.equalsIgnoreCase(t)
```

- Ma

```
boolean regionMatches(boolean
ignoreCase, int toffset, String
other, int ooffset, int len)
```

- Perché non regionMatchesIgnoreCase?
- Problema molto comune nel codice degli studenti

🔄recap

- Processi e strumenti per lo sviluppo
  - Git
  - Unit test/Extreme programming
- Java di base ed avanzato
  - Tipi di interfacce, polimorfismo e ereditarietà
  - Classi interne
  - Collezioni
- Programmazione dell'interfaccia utente
  - Costruzione di applicazioni Swing
  - Gestione degli eventi
- Progettazione orientata agli oggetti
  - Diagrammi UML
  - Pattern di programmazione
  - Programmazione funzionale e streaming

Credits

Queste slide sono una personale rielaborazione delle slide del prof. Horstmann